MyCodeSchool

Exerciții — algoritmi pe șiruri de caractere

Recapitulare · C++ · 12 algoritmi, 60 probleme · David

`algoritmi.h` **e doar de studiu.** Are înăuntru algoritmii de mai jos, gata scriși. Îl citești, îl închizi, apoi **scrii tu** algoritmul în headerul tău. Dacă te blochezi, îl trasezi pe hârtie, nu-l copiezi.

Cum lucrezi la fișă:

- **pentru fiecare algoritm îți faci headerul lui** — numele e scris în titlul fiecărei secțiuni;
- sus în header scrii **tu** algoritmul de bază, apoi cele 5 funcții, care îl folosesc;
- garda se scrie fără cratimă: `PARCURGERE_LUNGIME_H_INCLUDED`, nu `PARCURGERE-LUNGIME_H_INCLUDED`;
- șirurile sunt `char s[256]` (cel mult 255 de caractere); în secțiunile cu cuvinte (8, 9, 12): cel mult 50 de cuvinte, fiecare de cel mult 30 de litere, iar rezultatul are cel mult 255 de caractere; listele de cuvinte sunt `char c[50][31]`;
- o funcție `void` care primește `s` îl **modifică pe loc**; la funcțiile cu `rez` rezultatul se scrie în `rez`; în rest funcția **întoarce** rezultatul, iar `cout` apare doar în `main`;
- în exemple, **ieșirea** este valoarea întoarsă sau șirul (`s`, `rez`) după apel;
- în `main.cpp` incluzi headerul la care lucrezi, apoi chemi funcțiile pe exemplele din fișă;
- deschizi proiectul din folderul lecției în Code::Blocks și faci Build and run (**F9**) înainte de fiecare commit.

Cum citești exemplele:

- un șir e scris între ghilimele, ca să se vadă spațiile de la capete și cele duble; **ghilimelele nu fac parte din șir**; `""` este șirul vid;
- un caracter e scris între apostrofuri: `'a'`; o listă de cuvinte e scrisă între acolade: `{ana, mere}` (n = numărul de cuvinte);
- la secțiunea 12, `\n` înseamnă Enter: sfârșitul unei linii pe care o tastezi (și ultima linie se termină cu Enter).

Reguli:

- **Nu ai voie** să incluzi `algoritmi.h`, să chemi ceva din el sau să copiezi din el.
- **Ai voie** să chemi funcțiile **tale**, din headerele tale, oricâte, în orice combinație.
- Doar `char[]` și `<cstring>` (`strlen`, `strcpy`, `strcat`, `strcmp`, `strchr`, `strstr`, `strtok`); textul se citește cu `cin.getline(s, 256)`.
- **Nu ai voie:** `string`, `vector` din bibliotecă, `#include <algorithm>`, `atoi`, `to_string`; literele se recunosc după codul ASCII, fără `isalpha` sau `toupper`.
- Un vector de frecvență (`int f[26]`) ai voie începând cu secțiunea 3.

1. Parcurgerea șirului · `parcurgere-lungime.h`

Studiezi din `algoritmi.h`: `lungime, pozitiePrimulSpatiu`

Câte spații 1.1

Scrie o funcție care întoarce câte spații are șirul.

`int numarSpatii(char s[])`

```
"Ana are mere"  ->  2
""              ->  0
"   "           ->  3
"a"             ->  0
" ab "          ->  2
```

Ultima poziție

1.2

Scrie o funcție care întoarce poziția ultimei apariții a caracterului c în șir, numărată de la 0, sau -1 dacă nu apare.

`int ultimaPozitie(char s[], char c)`

```
s = "banana" , c = 'a'  ->  5
s = "banana" , c = 'b'  ->  0
s = "banana" , c = 'z'  ->  -1
s = "" , c = 'a'        ->  -1
s = "a" , c = 'a'       ->  0
```

Cel mai lung bloc

1.3

Scrie o funcție care întoarce lungimea celui mai lung bloc de caractere identice vecine.

`int celMaiLungBloc(char s[])`

```
"aabbbc"  ->  3
"abc"     ->  1
""        ->  0
"aaaa"    ->  4
"a"       ->  1
```

Șir crescător

1.4

Scrie o funcție care spune dacă fiecare caracter are codul ASCII cel puțin egal cu al celui dinaintea lui.

`bool esteCrescator(char s[])`

```
"abcc"  ->  true
"abca"  ->  false
""      ->  true
"z"     ->  true
"aA"    ->  false
```

Primul diferit

1.5

Scrie o funcție care întoarce poziția primului caracter diferit de primul caracter al șirului, sau -1 dacă nu există.

`int primulDiferit(char s[])`

```
"aaabaa"  ->  3
"aaaa"    ->  -1
""        ->  -1
"a"       ->  -1
"ab"      ->  1
```

2. Litere mari, litere mici · `litere-clasificare.h`

Studiezi din `algoritmi.h`: `esteLitera, esteCifra, esteVocala, inMajuscula, inMinuscula, majuscule`

Inițiale mari — 2.1

Scrie o funcție care schimbă pe loc șirul astfel încât fiecare cuvânt să înceapă cu literă mare, iar restul literelor lui să fie mici.

`void initialeMari(char s[])`

```
"ana ARE mere"  ->  "Ana Are Mere"
""              ->  ""
" ab"           ->  " Ab"
"x"             ->  "X"
"3 ani"         ->  "3 Ani"
```

Schimbă cazul — 2.2

Scrie o funcție care transformă pe loc fiecare literă mică în literă mare și fiecare literă mare în literă mică; restul caracterelor rămân la fel.

`void schimbaCazul(char s[])`

```
"Ana 3x"  ->  "aNA 3X"
"aBc Z"   ->  "AbC z"
""        ->  ""
"z"       ->  "Z"
"123"     ->  "123"
```

Consoane — 2.3

Scrie o funcție care întoarce câte consoane are șirul, adică literele (mici sau mari) care nu sunt vocale.

`int numarConsoane(char s[])`

```
"Ana are 3 Mere"  ->  4
""                ->  0
"aeiou"           ->  0
"xyz 123"         ->  3
"BCD"             ->  3
```

Doar cifrele — 2.4

Scrie o funcție care lasă pe loc în șir doar caracterele care sunt cifre, în ordinea inițială.

`void pastreazaCifrele(char s[])`

```
"a1b22c"  ->  "122"
"abc"     ->  ""
""        ->  ""
"7"       ->  "7"
"  1 2 "  ->  "12"
```

Predominant

2.5

Scrie o funcție care întoarce 'M' dacă șirul are mai multe litere mari decât mici, 'm' dacă are mai multe litere mici, '=' dacă sunt tot atâtea.

`char tipPredominant(char s[])`

```
"Ana Are MERE"  ->  'M'
"abC"           ->  'm'
"aB"            ->  '='
""              ->  '='
"123"           ->  '='
```

3. Aparițiile și frecvența literelor · `frecventa-litere.h`

Studiezi din `algoritmi.h` : `aparitii, frecventaLitere`

Litere diferite

3.1

Scrie o funcție care întoarce câte litere diferite apar într-un șir format din litere mici și spații.

`int litereDiferite(char s[])`

```
"mama"     ->  2
""         ->  0
"a"        ->  1
"abc abc"  ->  3
"aaa bbb"  ->  2
```

Litera frecventă

3.2

Scrie o funcție care întoarce litera mică ce apare de cele mai multe ori; la egalitate, pe cea mai mică alfabetic; dacă șirul nu are litere, întoarce '?'.

`char literaFrecventa(char s[])`

```
"banana"  ->  'a'
"abab"    ->  'a'
""        ->  '?'
"z"       ->  'z'
"  "      ->  '?'
"xxyy"    ->  'x'
```

Anagrame

3.3

Scrie o funcție care primește două șiruri de litere mici și spune dacă au aceleași litere, fiecare apărând de același număr de ori.

`bool anagrame(char a[], char b[])`

```
a = "lac" , b = "cal"    ->  true
a = "ab" , b = "abb"     ->  false
a = "" , b = ""          ->  true
a = "aab" , b = "abb"    ->  false
a = "rosa" , b = "sora"  ->  true
```

Litere unice

3.4

Scrie o funcție care întoarce câte litere apar o singură dată într-un șir format din litere mici și spații.

`int litereUnice(char s[])`

```
"banana"  ->  1
"aabb"    ->  0
""        ->  0
"abc"     ->  3
"a b a"   ->  1
```

Prima literă lipsă

3.5

Scrie o funcție care întoarce cea mai mică literă de la 'a' la 'z' care nu apare în șir, sau '-' dacă apar toate.

`char primaLipsa(char s[])`

```
"abcxyz"                                    ->  'd'
"bcd"                                       ->  'a'
""                                          ->  'a'
"the quick brown fox jumps over the lazy dog"  ->  '-'
"abc"                                       ->  'd'
```

4. Inversarea și palindromul · `inversare-palindrom.h`

Studiezi din `algoritmi.h`: `inverseaza, estePalindrom`

Lipește oglinditul

4.1

Scrie o funcție care adaugă la sfârșitul șirului oglinditul lui.

`void lipesteOglinditul(char s[])`

```
"abc"   ->  "abccba"
""      ->  ""
"a"     ->  "aa"
"ab c"  ->  "ab cc ba"
"aa"    ->  "aaaa"
```

Palindrom fără spații

4.2

Scrie o funcție care spune dacă șirul rămâne palindrom după ce ignori spațiile.

`bool palindromFaraSpatii(char s[])`

```
"ora ro"   ->  true
"ana are"  ->  false
""         ->  true
" "        ->  true
"a ba"     ->  true
```

Prefix palindrom

4.3

Scrie o funcție care întoarce lungimea celui mai lung prefix al șirului care este palindrom.

`int prefixPalindrom(char s[])`

```
"abacd"  ->  3
"abc"    ->  1
""       ->  0
"aaa"    ->  3
"abcba"  ->  5
```

Inversează secvența 4.4

Scrie o funcție care inversează pe loc caracterele de pe pozițiile i, i + 1, ..., j (numărate de la 0), cu 0 <= i <= j < lungimea șirului.

`void inverseazaSecventa(char s[], int i, int j)`

```
s = "abcdef" , i = 1 , j = 3  ->  "adcbef"
s = "abcdef" , i = 0 , j = 5  ->  "fedcba"
s = "abcdef" , i = 2 , j = 2  ->  "abcdef"
s = "a" , i = 0 , j = 0       ->  "a"
s = "abcdef" , i = 4 , j = 5  ->  "abcdfe"
```

Aproape palindrom 4.5

Scrie o funcție care spune dacă șirul devine palindrom prin ștergerea a cel mult un caracter.

`bool palindromFaraUnul(char s[])`

```
"abca"     ->  true
"abcd"     ->  false
""         ->  true
"ab"       ->  true
"abccbxa"  ->  true
```

5. Ștergerea și inserarea · `stergere-inserare.h`

Studiezi din `algoritmi.h`: `stergeCaracter, insereazaCaracter`

Fără vocale 5.1

Scrie o funcție care șterge din șir, pe loc, toate vocalele, mici și mari.

`void faraVocale(char s[])`

```
"Ana are mere"  ->  "n r mr"
""              ->  ""
"aeiou"         ->  ""
"xyz"           ->  "xyz"
"Eu"            ->  ""
```

Spații curate 5.2

Scrie o funcție care șterge spațiile de la începutul și de la sfârșitul șirului și lasă un singur spațiu între cuvinte.

`void normalizeazaSpatii(char s[])`

```
"  ana   are  mere "  ->  "ana are mere"
"   "                 ->  ""
""                    ->  ""
"a"                   ->  "a"
"a  b"                ->  "a b"
```

Vocale dublate

5.3

Scrie o funcție care scrie pe loc fiecare vocală de două ori la rând.

`void dubleazaVocale(char s[])`

```
"ana"  ->  "aanaa"
""     ->  ""
"xyz"  ->  "xyz"
"e"    ->  "ee"
"Eu"   ->  "EEuu"
```

Fără vecine egale

5.4

Scrie o funcție care înlocuiește pe loc fiecare grup de caractere identice vecine cu un singur caracter.

`void faraVecineEgale(char s[])`

```
"aabbbc"  ->  "abc"
""        ->  ""
"a"       ->  "a"
"abab"    ->  "abab"
"aaaa"    ->  "a"
```

După fiecare apariție

5.5

Scrie o funcție care inserează caracterul d imediat după fiecare apariție a caracterului c din șirul inițial.

`void insereazaDupa(char s[], char c, char d)`

```
s = "banana" , c = 'a' , d = '-'  ->  "ba-na-na-"
s = "abc" , c = 'z' , d = '-'     ->  "abc"
s = "" , c = 'a' , d = 'b'        ->  ""
s = "aa" , c = 'a' , d = 'a'      ->  "aaaa"
s = "a" , c = 'a' , d = 'b'       ->  "ab"
```

6. Copiere, concatenare, comparare · `copiere-comparare.h`

Studiezi din `algoritmi.h`: `concateneaza, esteInainte, ordoneazaCuvinte`

Cu spațiu între ele

6.1

Scrie o funcție care pune în rez șirul a, un spațiu și șirul b; dacă unul dintre ele este vid, rez este doar celălalt.

`void lipesteCuSpatiu(char a[], char b[], char rez[])`

```
a = "Ana" , b = "Pop"  ->  "Ana Pop"
a = "" , b = "Pop"     ->  "Pop"
a = "Ana" , b = ""     ->  "Ana"
a = "" , b = ""        ->  ""
a = "a" , b = "b"      ->  "a b"
```

Care vine primul

6.2

Scrie o funcție care întoarce 1 dacă a vine înaintea lui b în ordine lexicografică, 2 dacă b vine înaintea lui a, 0 dacă sunt egale.

`int primulInOrdine(char a[], char b[])`

```
a = "ana" , b = "mere"      ->  1
a = "mere" , b = "ana"      ->  2
a = "ana" , b = "ana"       ->  0
a = "ana" , b = "anatomie"  ->  1
a = "" , b = "a"            ->  1
```

Primul în dicționar

6.3

Scrie o funcție care copiază în rez cuvântul care vine primul în ordine lexicografică dintre cele n cuvinte din c (n >= 1).

`void primulAlfabetic(char c[][31], int n, char rez[])`

```
{mere, pere, ana}  ->  "ana"
{x}                ->  "x"
{ab, a}            ->  "a"
{b, a, a}          ->  "a"
{Zeta, ana}        ->  "Zeta"
```

Scurtele întâi

6.4

Scrie o funcție care ordonează cuvintele din c crescător după lungime, iar la aceeași lungime lexicografic.

`void ordoneazaDupaLungime(char c[][31], int n)`

```
{pere, ana, mere, fi}  ->  {fi, ana, mere, pere}
{b, a}                 ->  {a, b}
{abc, ab, a}           ->  {a, ab, abc}
{x}                    ->  {x}
{}                     ->  {}
```

Cuvinte distincte

6.5

Scrie o funcție care întoarce câte cuvinte diferite are lista c.

`int cuvinteDistincte(char c[][31], int n)`

```
{ana, mere, ana}  ->  2
{}                ->  0
{a}               ->  1
{a, a, a}         ->  1
{a, b, c}         ->  3
```

7. Căutarea și înlocuirea · `cautare-inlocuire.h`

Studiezi din `algoritmi.h` : `pozitieCaracter, pozitieSubsir, inlocuiestePrima`

Câte apariții 7.1

Scrie o funcție care întoarce de câte ori apare t în s, inclusiv aparițiile care se suprapun.

`int aparitiiSubsir(char s[], char t[])`

```
s = "banana" , t = "ana"  ->  2
s = "aaaa" , t = "aa"     ->  3
s = "abc" , t = "abcd"    ->  0
s = "abcab" , t = "ab"    ->  2
s = "abc" , t = "z"       ->  0
```

Ultima apariție 7.2

Scrie o funcție care întoarce poziția de început a ultimei apariții a lui t în s, sau -1 dacă nu apare.

`int ultimaAparitie(char s[], char t[])`

```
s = "abcab" , t = "ab"  ->  3
s = "abc" , t = "z"     ->  -1
s = "ab" , t = "ab"     ->  0
s = "a" , t = "ab"      ->  -1
s = "aaaa" , t = "aa"   ->  2
```

Înlocuiește tot 7.3

Scrie o funcție care înlocuiește pe loc fiecare apariție a lui vechi cu nou, de la stânga la dreapta; ce pui în loc nu se mai caută din nou.

`void inlocuiesteToate(char s[], char vechi[], char nou[])`

```
s = "banana" , vechi = "an" , nou = "AN"  ->  "bANANa"
s = "abab" , vechi = "ab" , nou = "x"     ->  "xx"
s = "aaa" , vechi = "a" , nou = "aa"      ->  "aaaaaa"
s = "abc" , vechi = "z" , nou = "y"       ->  "abc"
s = "ab" , vechi = "ab" , nou = ""        ->  ""
```

Șterge subșirul 7.4

Scrie o funcție care șterge din s prima apariție a lui t și repetă cât timp t mai apare în s.

`void stergeSubsir(char s[], char t[])`

```
s = "abcabc" , t = "bc"  ->  "aa"
s = "ababc" , t = "abc"  ->  "ab"
s = "xx" , t = "x"       ->  ""
s = "abc" , t = "z"      ->  "abc"
s = "aabb" , t = "ab"    ->  ""
```

Cuvânt întreg

7.5

Scrie o funcție care întoarce de câte ori apare t în s ca cuvânt întreg, adică încadrat de spații sau de capetele șirului.

`int aparitiiCuvant(char s[], char t[])`

```
s = "ana are ana si anaconda" , t = "ana"  ->  2
s = "ana" , t = "ana"                      ->  1
s = "banana" , t = "ana"                   ->  0
s = "" , t = "a"                           ->  0
s = "ana ana ana" , t = "ana"              ->  3
```

8. Cuvinte cu strtok · `cuvinte-strtok.h`

Studiezi din `algoritmi.h` : `numarCuvinte, imparteCuvinte`

Câte propoziții

8.1

Scrie o funcție care întoarce câte propoziții are textul, propozițiile fiind separate prin '.', '!' sau '?' (numeri fragmentele care conțin cel puțin un caracter diferit de spațiu).

`int numarPropozitii(char s[])`

```
"Plec acasa. Vii si tu? Da! "  ->  3
""                             ->  0
"..."                          ->  0
"Fara punct"                   ->  1
"a.. b."                       ->  2
```

Al k-lea cuvânt

8.2

Scrie o funcție care copiază în rez al k-lea cuvânt din s, cuvintele fiind separate prin caracterele din sep; dacă nu există, rez este vid.

`void alKlea(char s[], char sep[], int k, char rez[])`

```
s = "ana are, mere" , sep = " ," , k = 2  ->  "are"
s = "ana are, mere" , sep = " ," , k = 4  ->  ""
s = "" , sep = " " , k = 1                ->  ""
s = "a" , sep = " " , k = 1               ->  "a"
s = "a;b;c" , sep = ";" , k = 3           ->  "c"
```

Acronimul

8.3

Scrie o funcție care pune în rez prima literă a fiecărui cuvânt din s (cuvintele sunt separate prin spații), în ordine.

`void acronim(char s[], char rez[])`

```
"Universitatea Tehnica Cluj"  ->  "UTC"
""                            ->  ""
"  ana  "                     ->  "a"
"a"                           ->  "a"
"a b  c"                      ->  "abc"
```

Cuvintele pe dos

8.4

Scrie o funcție care scrie pe loc cuvintele șirului în ordine inversă, separate prin câte un spațiu.

`void inverseazaCuvintele(char s[])`

```
"ana are mere"  ->  "mere are ana"
"a"             ->  "a"
""              ->  ""
"  a  b "       ->  "b a"
"x y"           ->  "y x"
```

Cuvinte unice

8.5

Scrie o funcție care întoarce câte cuvinte apar o singură dată în șir (cuvintele sunt separate prin spații).

`int cuvinteUnice(char s[])`

```
"ana are ana mere"  ->  2
""                  ->  0
"a a"               ->  0
"a b"               ->  2
"a b a b c"         ->  1
```

9. Cuvântul cel mai lung · `cuvantul-cel-mai-lung.h`

Studiezi din `algoritmi.h`: `lungimeMaxima`

Cel mai lung cuvânt

9.1

Scrie o funcție care copiază în rez cel mai lung cuvânt din s; la egalitate, pe primul; dacă nu sunt cuvinte, rez este vid.

`void celMaiLungCuvant(char s[], char rez[])`

```
"ana are mere"  ->  "mere"
"ab cd"         ->  "ab"
""              ->  ""
"a"             ->  "a"
"  x  yy  "     ->  "yy"
```

Cel mai scurt

9.2

Scrie o funcție care întoarce lungimea celui mai scurt cuvânt din s, sau 0 dacă șirul nu are cuvinte.

`int lungimeCeluiMaiScurt(char s[])`

```
"ana are mere"  ->  3
""              ->  0
"a bb"          ->  1
"  abc  "       ->  3
"xx y zzz"      ->  1
```

Cuvinte de lungime k

9.3

Scrie o funcție care întoarce câte cuvinte din s au exact k litere.

`int cuvinteDeLungime(char s[], int k)`

```
s = "ana are mere si tu" , k = 3  ->  2
s = "ana are mere si tu" , k = 4  ->  1
s = "" , k = 3                    ->  0
s = "ana are mere" , k = 9        ->  0
s = "a" , k = 1                   ->  1
```

Aceeași literă la capete

9.4

Scrie o funcție care întoarce câte cuvinte încep și se termină cu aceeași literă (un cuvânt de o literă se numără).

`int acelasiCapat(char s[])`

```
"ana are mere eve"  ->  2
"a b"               ->  2
""                  ->  0
"ab ba"             ->  0
"Ana ana"           ->  1
```

Cele mai multe vocale

9.5

Scrie o funcție care copiază în rez cuvântul cu cele mai multe vocale; la egalitate, pe primul; dacă nu sunt cuvinte, rez este vid.

`void maiMulteVocale(char s[], char rez[])`

```
"ana are mere"   ->  "ana"
"bcd aeiou ab"   ->  "aeiou"
""               ->  ""
"xyz qrs"        ->  "xyz"
"Ana are Eugen"  ->  "Eugen"
```

10. Din șir în număr și înapoi · `sir-numar.h`

Studiezi din `algoritmi.h` : `sirInNumar, numarInSir`

Suma cifrelor din șir

10.1

Scrie o funcție care întoarce suma cifrelor care apar într-un șir de litere și cifre.

`int sumaCifreDinSir(char s[])`

```
"a1b2c3"  ->  6
""        ->  0
"abc"     ->  0
"9"       ->  9
"x99"     ->  18
```

Cel mai mare din cifrele șirului

10.2

Scrie o funcție care întoarce cel mai mare număr care se poate forma cu cifrele din șir (cel mult 9 cifre); fără cifre, întoarce 0.

`int celMaiMareDinCifre(char s[])`

```
"a3b9c1"  ->  931
"abc"     ->  0
"5"       ->  5
""        ->  0
"1020"    ->  2100
```

În baza 2

10.3

Scrie o funcție care pune în rez numărul n scris în baza 2.

`void inBinar(int n, char rez[])`

```
10   ->  "1010"
0    ->  "0"
1    ->  "1"
255  ->  "11111111"
6    ->  "110"
```

Suma numerelor din text

10.4

Scrie o funcție care întoarce suma numerelor din text, un număr fiind un grup de cifre vecine.

`int sumaNumerelor(char s[])`

```
"a12b3 7"  ->  22
""         ->  0
"abc"      ->  0
"100"      ->  100
"12 12"    ->  24
"7x8"      ->  15
```

Adunarea numerelor mari

10.5

Scrie o funcție care pune în rez suma a două numere naturale date ca șiruri de cifre, fără zerouri la început și cu cel mult 100 de cifre.

`void adunaMari(char a[], char b[], char rez[])`

```
a = "999" , b = "1"                                ->  "1000"
a = "0" , b = "0"                                  ->  "0"
a = "1234567890123456" , b = "9876543210987654"    ->  "11111111101111110"
a = "5" , b = "5"                                  ->  "10"
a = "123" , b = "9"                                ->  "132"
```

11. Cifrul lui Cezar · `cifrul-cezar.h`

Studiezi din `algoritmi.h` : `deplaseaza, cezar`

Decriptare 11.1

Scrie o funcție care decriptează pe loc un text criptat cu cifrul lui Cezar cu cheia k, adică literele au fost deplasate k poziții înainte; celelalte caractere rămân la fel.

`void decripteaza(char s[], int k)`

```
s = "Dqd duh phuh" , k = 3  ->  "Ana are mere"
s = "abc" , k = 0           ->  "abc"
s = "abc" , k = 3           ->  "xyz"
s = "" , k = 5              ->  ""
s = "a" , k = 26            ->  "a"
```

Cheie de substituție 11.2

Scrie o funcție care înlocuiește pe loc fiecare literă mică 'a' + i cu cheie[i]; celelalte caractere rămân la fel.

`void substituie(char s[], char cheie[])`

```
s = "abc" , cheie = "qwertyuiopasdfghjklzxcvbnm"    ->  "qwe"
s = "xyz" , cheie = "qwertyuiopasdfghjklzxcvbnm"    ->  "bnm"
s = "a b" , cheie = "qwertyuiopasdfghjklzxcvbnm"    ->  "q w"
s = "" , cheie = "qwertyuiopasdfghjklzxcvbnm"       ->  ""
s = "Ana 7" , cheie = "qwertyuiopasdfghjklzxcvbnm"  ->  "Afq 7"
```

Fiecare cuvânt cu cheia lui 11.3

Scrie o funcție care criptează pe loc fiecare cuvânt cu cifrul lui Cezar, cheia fiecărui cuvânt fiind lungimea lui.

`void cezarPeCuvinte(char s[])`

```
"ab cde"  ->  "cd fgh"
"xyz"     ->  "abc"
"a"       ->  "b"
""        ->  ""
"  hi  "  ->  "  jk  "
```

Literele ca numere 11.4

Scrie o funcție care pune în rez numerele de ordine din alfabet ale literelor mici din s (a = 1, ..., z = 26), separate prin câte un spațiu; celelalte caractere se ignoră.

`void pozitiiLitere(char s[], char rez[])`

```
"abz"  ->  "1 2 26"
"a b"  ->  "1 2"
""     ->  ""
"z"    ->  "26"
" "    ->  ""
```

Rotire spre stânga

11.5

Scrie o funcție care rotește pe loc șirul spre stânga cu k poziții; k poate fi mai mare decât lungimea șirului.

`void rotesteStanga(char s[], int k)`

```
s = "abcde" , k = 2  ->  "cdeab"
s = "abcde" , k = 7  ->  "cdeab"
s = "abcde" , k = 0  ->  "abcde"
s = "" , k = 3       ->  ""
s = "a" , k = 5      ->  "a"
s = "abcde" , k = 5  ->  "abcde"
```

12. Citirea cu getline · `citire-getline.h`

Studiezi din `algoritmi.h` : `citesteLinie, citesteNumar`

Linia mai lungă

12.1

Scrie o funcție care citește două linii și copiază în rez linia mai lungă; la egalitate, pe prima.

`void liniaMaiLunga(char rez[])`

```
"ana\nmere"         ->  "mere"
"ab\ncd"            ->  "ab"
"\nx"               ->  "x"
"un ban\nopt bani"  ->  "opt bani"
"x\n\nz"            ->  "x"
```

Linii cu o literă

12.2

Scrie o funcție care citește un număr n, apoi n linii, și întoarce câte linii conțin caracterul c.

`int liniiCuLitera(char c)`

```
intrare = "3\nana\nbob\nmama" , c = 'a'  ->  2
intrare = "0" , c = 'a'                  ->  0
intrare = "1\nbob" , c = 'b'             ->  1
intrare = "2\nx y\nyy" , c = 'y'         ->  2
intrare = "2\nab\nab" , c = 'z'          ->  0
```

Până la linia goală

12.3

Scrie o funcție care citește linii până la prima linie goală (sau până la sfârșitul intrării) și întoarce câte linii a citit înainte de ea.

`int numaraPanaLaGoala()`

```
"a\nb\n\nc"  ->  2
""           ->  0
"a\nb"       ->  2
"ab cd\n"    ->  1
"a\n \nb"    ->  3
```

Cuvântul lung dintr-o linie

12.4

Scrie o funcție care citește o linie și întoarce lungimea celui mai lung cuvânt din ea.

`int lungimeCuvantLung()`

```
"ana are mere"  ->  4
""              ->  0
"  x  "         ->  1
"abc"           ->  3
"a bb ccc dd"   ->  3
```

Cel mai lung cuvânt din text

12.5

Scrie o funcție care citește linii până la prima linie goală (sau până la sfârșitul intrării) și copiază în rez cel mai lung cuvânt din tot textul; la egalitate, pe primul.

`void celMaiLungDinText(char rez[])`

```
"ana are\nmere multe\n\nzzzzzzzzz"  ->  "multe"
""                                  ->  ""
"a\nbb\nccc"                        ->  "ccc"
"ab cd\nef"                         ->  "ab"
"\nabc"                             ->  ""
```

Pentru fiecare funcție pe care o scrii, verific-o pe:

1. un șir **vid** (`""`) și un șir cu **un singur caracter**;
2. un șir cu **spații multiple** între cuvinte și cu **spații la capete**;
3. cazul în care ce cauți **nu există** în șir;
4. cazul în care ce cauți e chiar **la început** sau chiar **la sfârșitul** șirului.